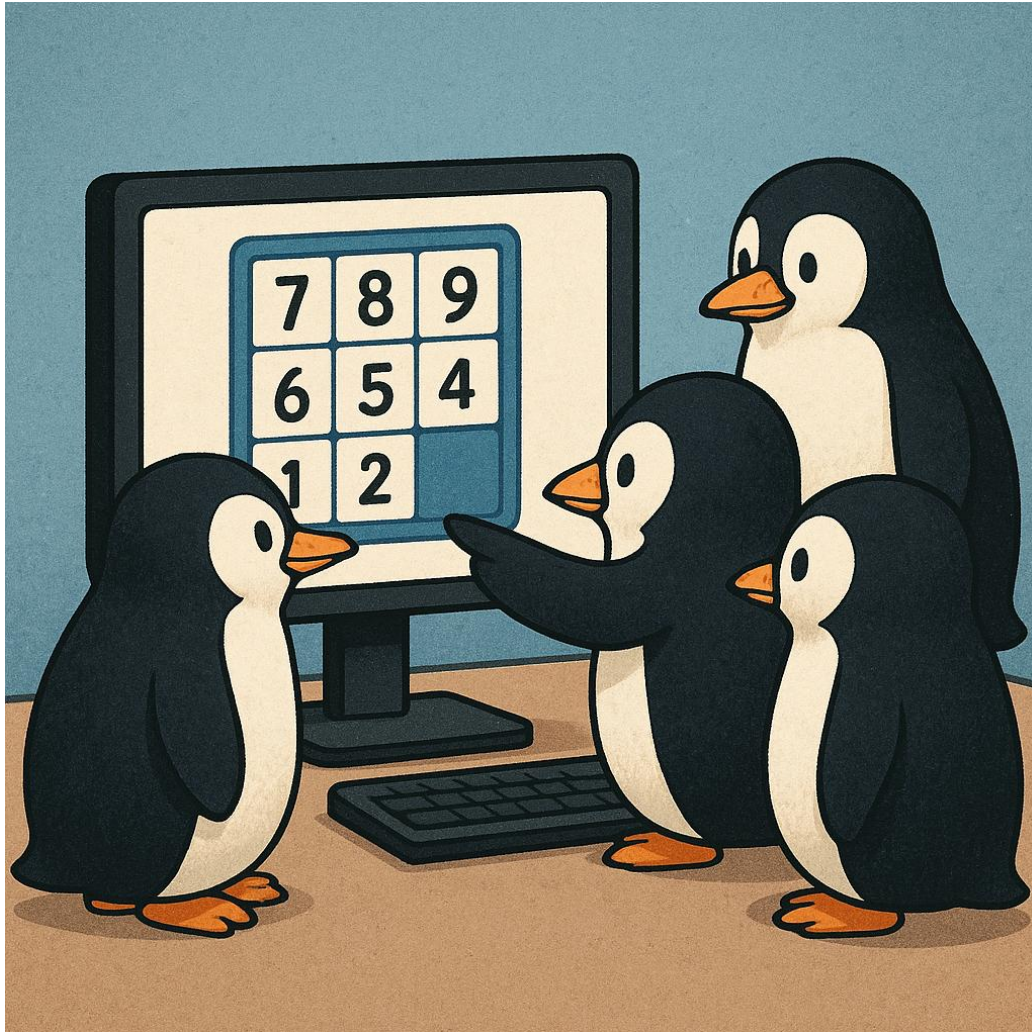


Introduction to AI and Data Science ILV

Exercise 1: 8-Puzzle



Presented by:

Group 4 – F
Ayden Kierein
Florian Koller
Sarah Paska
Vanessa Straka

Contents

1. Task Description: 8-Puzzle.....	2
2. Component Checklist	3
3. Task Documentation	4

1. Task Description: 8-Puzzle

Start: From a random start state

- Check for solvability
- Generate goal state
- Using at least 2 different heuristic functions
- Provide estimate of algorithms complexity
- Implementation in Python
- Measure memory effort (number of nodes expanded) and run time for each of 100 random states and each heuristics
- Provide mean and standard deviation of memory usage and execution time for each heuristics

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

2. Component Checklist

1. Setup:
 - ☒ Setup Python-Version and .venv
 - ☒ Requirements.txt
 - ☒ Project-structure clear
2. Check for Solvability
 - ☒ It's solvable
 - ☒ Goal state is reliably achievable
3. Implement two Heuristics:
 - ☒ Hamming
 - ☒ Manhattan
 - ☒ Provide an estimate of both algorithm complexity
4. Compare two Heuristics (using 100 random searches each):
 - ☒ Hamming: 100 searches done
 - ☒ Manhattan: 100 searches done
 - ☒ Memory Usage (number of expanded nodes in the search tree)
 - ☒ Computation (Run) Time
5. Statistics for each Heuristic:
 - ☒ Mean deviation of memory usage
 - ☒ Mean deviation of execution time
 - ☒ Standard deviation of memory usage
 - ☒ standard deviation of execution time
6. Comment Code, provide for each submodule at least:
 - ☒ What are Inputs and Outputs
 - ☒ What is the Function of the submodule
7. Don't just copy/paste code from Internet
 - ☒ Provide your own structure
 - ☒ Comment the code (see above)
 - ☒ Measure space (memory) and time complexity of each heuristics

3. Task Documentation

1. Short task description

The goal of this assignment was to implement and analyze an A* search solver for the 8-puzzle (3×3 sliding puzzle). The program can solve any solvable random puzzle automatically using two different heuristics:

- a. **Manhattan distance:** total number of moves each tile is away from its goal position
- b. **Hamming distance:** number of misplaced tiles (excluding the blank)

The solver finds the optimal path from a start state to the goal state, while the GUI visualizes the puzzle, lets the user play manually, and can run 100-board benchmarks to compare both heuristics in terms of runtime and nodes expanded.

2. Software architecture diagram

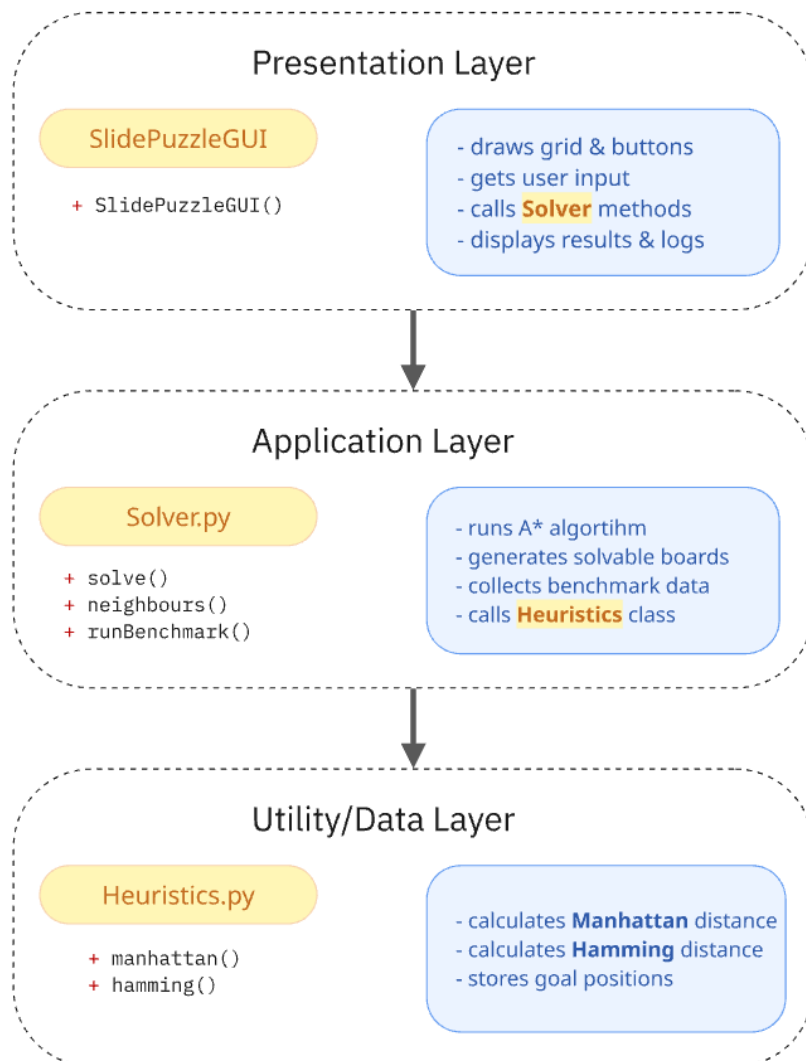


Figure 1: SAD 8-Puzzle by Group4F

3. Short descriptions of modules and interfaces

Module	gui.py	
Main Component	SlidePuzzleGUI	
Responsibilities	- Handles all user interactions via the Tkinter interface - Displays the 3×3 grid, moves tiles, and shows results of the solver - Starts solver and benchmark processes through button events	
Methods	create_widgets()	Builds the full GUI layout (grid, control panel, log area)
	log_message(message, ...)	Writes a timestamped message into the log window
	shuffle_tiles()	Generates new random solvable board, updates display
	move_tile(i, j)	Moves a tile into the empty space if the move is valid
	check_win()	Checks whether current puzzle state == goal state
	reset_game()	Resets all variables, generates a new shuffled board
	update_buttons()	Updates button texts, colors to match current tile layout
	calculate_heuristics()	Computes/displays current Manhattan Hamming values
	update_timer()	Updates the displayed search time while the solver runs
	get_current_state_2d()	Converts internal 1D tile list into 2D tuple state for solver
	set_current_state_from_2d(state_2d)	Loads a 2D state (from solver) back into GUI
	run_single_benchmark(heuristic)	Runs 100-board benchmark for selected heuristic
	find_and_show_solution(heuristic)	Runs A* solver with heuristic and animates solution path
	animate_solution()	Step-by-step animation of solution path on board

Module	solver.py	
Main Component	Solver	
Responsibilities	- Implements the A* search algorithm for the 8-puzzle - Handles state expansion, cost calculation, and benchmarking - Calls Heuristics class to compute the heuristic values for each state	
Methods	flatten(state)	Flattens a 2D tuple state into a 1D list of tiles
	generateRandomSolvableBoard()	Creates a random solvable 3×3 puzzle configuration
	isSolvable(state)	Checks if a flattened board configuration is solvable
	neighbors(state)	Generates all valid neighbor states by sliding the blank
	calculateCosts(state, g, heuristic)	Calculates total A* cost $f = g + h$ for a given state
	solve(startState, goalState, heuristic)	Performs A* search, returns path and expanded node count
	runBenchmark()	Runs 100 random boards for both heuristics and logs mean/stddev results

Module	heuristics.py	
Main Component	Heuristics	
Responsibilities	- Contains heuristic functions used by A* - Precomputes goal tile coordinates for fast lookup - Contains no dependency on Solver or GUI	
Methods	__init__(goalState)	Stores goal positions of each tile for quick lookup.
	_validate_state(state)	Checks that a given state matches the expected 3×3 dimensions
	manhattan(state)	Calculates total Manhattan distance for all tiles
	hamming(state)	Counts how many tiles are misplaced

Module interactions:
SlidePuzzleGUI → Solver
GUI calls <code>solver.solve()</code> or <code>solver.runBenchmark()</code> when user presses “Solve” or “Benchmark.” The solver returns the solution path and nodes expanded, which the GUI animates and logs.
Solver → Heuristics
For each state during the A* search, the solver calls either: <code>heuristic.manhattan(state)</code> or <code>heuristic.hamming(state)</code> to compute the heuristic cost $h(n)$.
Return Flow
After the solver finishes, the GUI displays the <i>runtime</i> , <i>moves count</i> , and <i>expanded nodes</i> in the text log.

Interfaces Between Layers:		
Direction	From → To	Type of Interaction
GUI → Solver	Function Calls	GUI triggers solving/benchmarking functions and passes selected heuristic
Solver → Heuristics	Function Calls	Solver uses heuristic functions to evaluate each board state
Solver → GUI	Return Values	Solver returns solution path/runtime/statistics
Heuristics → (none)	/	Heuristic module is independent, does not call any higher-level functions

4. Explain design decisions

The project was designed with a **three-layer structure**:

- 1) graphical user interface (GUI),
- 2) a solver (A* algorithm),
- 3) and a heuristics module (manhattan/hamming).

This separation makes the code easier to understand, test, and extend.

Python was chosen because it is simple and has all required libraries built-in. The *Tkinter* library was used for the interface since it is lightweight and included in Python by default.

The **A* algorithm** was used because it guarantees the shortest solution when paired with admissible heuristics. Both **Manhattan** and **Hamming** were implemented as demanded.

Puzzle states are stored as tuples of tuples, which makes them immutable and fast to compare or store in sets. A helper function converts between 1D and 2D layouts for the solver and GUI.

During development, the focus was on keeping the program easy to read and modular, so that future extensions (like new heuristics or bigger puzzles) can be added easily.

This design also helped in debugging and testing, since each module can be verified separately before integrating everything together.

5. Discussion and conclusions

- **Describe your experience**

Developing this project was a good learning experience for understanding how search algorithms work in practice. Implementing the A* algorithm helped us see how heuristics directly influence both performance and memory usage.

- **Provide a table with complexity comparisons of different heuristics**

Time Complexity: The time complexity of an algorithm measures how the running time increases as a function of the input size. In simple terms, it shows how the time needed to execute an algorithm grows when the amount of input data becomes larger.

O(N²) Complexity: This is also called Quadratic Time Complexity. In this case, the running time of the algorithm increases proportionally to the square of the input size. For large inputs, this can result in very long execution times. The letter O in O(N²) represents the *worst-case scenario* for how the algorithm's running time scales.

Hamming: O(n²)	• For an n×n puzzle (e.g., 3×3 → n=3), you have n² tiles • The function checks each tile once to see if it's misplaced → linear in the number of tiles → O(n²) • This is very fast
Manhattan: O(n²)	• Also checks each tile once but performs two subtractions and two absolute values (row and column differences) • Slightly slower in constant time (not in order of growth)
A* Search: O(b^d)	• Here, <i>b</i> = branching factor (⇒ how many possible moves from each state, typically up to 4). • <i>d</i> = depth of the solution (number of moves in the shortest path) • This means the number of states grows exponentially with <i>d</i>. • That's why A* can become slow for bigger puzzles • If heuristics are good, A* explores much fewer nodes, reducing the <i>effective branching factor</i>.

Space Complexity: The space complexity of an algorithm measures how much memory or storage space it requires as a function of the input size. It includes both the memory used by variables and data structures, as well as any additional temporary space needed during execution. In simple terms, it describes how the memory usage grows when the size of the input increases.

O(1) Complexity: This is also called Constant Space Complexity. It means the algorithm uses a fixed amount of memory, regardless of the input size. The space required does not increase even if the amount of input data becomes larger. For example, when only a few counters or variables are used to process data one piece at a time, the space complexity is O(1).

Hamming / Manhattan: O(1)	• They don't store large data structures; just compute a number → constant memory
A* Search: O(b^d)	• A* keeps all visited and frontier (open list) states in memory. • As the tree grows exponentially with <i>d</i> (<i>depth of solution</i>), A* memory can grow very fast (memory is often the biggest problem)

Complexity Comparison Table:			
	Hamming Distance	Manhattan Distance	A* Search
Time complexity (per state)	$O(n^2)$	$O(n^2)$	$O(b^d)$
Space Complexity	$O(1)$	$O(1)$	$O(b^d)$
Description	Counts misplaced tiles	Sums distances of each tile from its goal position	Explores nodes based on $f(n)=g(n)+h(n)$; complete and optimal if $h(n)$ is admissible
Observed Performance	Simple but less informative, expands more nodes during A*	More accurate and efficient in A*	efficiency depends heavily on heuristic accuracy

Memory Usage:

```
2025-10-25 15:22:17 - Heuristic: manhattan, Mean nodes: 1515.00, Standard nodes: 1788.72
2025-10-25 15:22:17 - Heuristic: hamming, Mean nodes: 15823.26, Standard nodes: 15629.15
2025-10-25 15:53:55 - Heuristic: manhattan, Mean nodes: 1485.02, Standard nodes: 1960.77
2025-10-25 15:53:55 - Heuristic: hamming, Mean nodes: 18949.12, Standard nodes: 17425.32
2025-10-25 20:24:27 - Heuristic: manhattan, Mean nodes: 1421.65, Standard nodes: 1708.55
2025-10-25 20:24:27 - Heuristic: hamming, Mean nodes: 13638.76, Standard nodes: 16482.63
2025-10-25 20:24:47 - Heuristic: manhattan, Mean nodes: 1664.38, Standard nodes: 1722.57
2025-10-25 20:24:47 - Heuristic: hamming, Mean nodes: 17799.58, Standard nodes: 19291.60
```

Runtime:

```
2025-10-25 15:22:17 - Heuristic: manhattan, Mean runtime: 0.0163 s, Standard runtime: 0.0201 s
2025-10-25 15:22:17 - Heuristic: hamming, Mean runtime: 0.1771 s, Standard runtime: 0.1920 s
2025-10-25 15:53:55 - Heuristic: manhattan, Mean runtime: 0.0144 s, Standard runtime: 0.0202 s
2025-10-25 15:53:55 - Heuristic: hamming, Mean runtime: 0.2000 s, Standard runtime: 0.1976 s
2025-10-25 20:24:27 - Heuristic: manhattan, Mean runtime: 0.0092 s, Standard runtime: 0.0115 s
2025-10-25 20:24:27 - Heuristic: hamming, Mean runtime: 0.0942 s, Standard runtime: 0.1222 s
2025-10-25 20:24:47 - Heuristic: manhattan, Mean runtime: 0.0108 s, Standard runtime: 0.0115 s
2025-10-25 20:24:47 - Heuristic: hamming, Mean runtime: 0.1241 s, Standard runtime: 0.1418 s
```

- **Possible improvements in future**

In the future, this project could be improved by adding more advanced heuristics and support for larger puzzles like the 15-puzzle. It would also be interesting to optimize the memory usage, since A* can use a lot of space for bigger boards.

The GUI could be made more fun with newer animations and live statistics. Lastly, running the benchmark in parallel and allowing the puzzle size to change dynamically would make the program faster and more flexible.